

Cite this: DOI: 00.0000/xxxxxxxxxx

An open-source, 3D-printed auger powder doser designed with generative artificial intelligence[†]

Sam Charles,^{*a} Will Mulberry,^a Luke Winters,^a Carl Robison,^a Gage Erickson,^a and Sterling G. Baird^{*a,b}

Received Date

Accepted Date

DOI: 00.0000/xxxxxxxxxx

Accurate, automated dispensing of many different powders is a recurring bottleneck for self-driving laboratories and high-throughput materials discovery, yet commercial automated powder dosers cost tens to hundreds of thousands of dollars. Here we present the base module of an open-source, low-cost powder doser: a 3D-printed Archimedes-auger dispenser with solenoid tapping and vibration assistance, servo-controlled tilt, and closed-loop gravimetric feedback from an analytical balance beneath the collection cup, controlled by a \$6 microcontroller. The complete single-channel module is built from printed parts and off-the-shelf components for under \$200. Uniquely, the platform's mechanical parts were designed without conventional CAD software: every part was modelled through an agent-mediated programmatic-CAD workflow—large-language-model (LLM) coding agents authoring parametric CAD code under human review, with chat-driven text-to-CAD services evaluated as comparators and one (Zoo Design Studio) adopted late in the project—and the full design history, including failures, is preserved in a public version-controlled repository. We report what worked and what did not: whole-assembly generation failed repeatedly, while a part-by-part workflow with explicit interface specifications, automated self-checks, and human visual review converged to printable, functional parts. We define a gravimetric dispensing-characterization protocol, implemented in the firmware, for evaluating a built channel across surrogate and target powders, and discuss the implications of low-cost powder dosing for autonomous additive-manufacturing alloy discovery. All design files, firmware, AI prompt/response logs, and the complete design history are openly available at <https://github.com/vertical-cloud-lab/powder-doser>.

1 Introduction

Self-driving laboratories (SDLs) couple automated experimentation hardware with machine-learning planners to close the loop between hypothesis and measurement, and have delivered order-of-magnitude accelerations in chemistry and materials discovery.^{1–4} A decade of digital-chemistry infrastructure has matured software orchestration,^{5,6} Bayesian experiment planning,⁷ and liquid-handling robotics, but *solid* handling remains comparatively underdeveloped: most SDL demonstrations either restrict themselves to liquids or rely on manual powder weighing as a hidden human-in-the-loop step.^{8,9} For autonomous alloy design by additive manufacturing (AM)—where candidate compositions are realized by blending elemental or master-alloy powders^{10–14}—accurate, repeatable, multi-powder dosing is the gat-

ing unit operation.

Commercial automated powder-dosing systems exist and perform well, but at a cost and form factor that excludes most academic SDLs. Gravimetric benchtop dispensers such as the METTLER TOLEDO XPR/CHRONECT line, Labman powder-dosing stations, and MTT's AM powder dispensers are priced from tens of thousands to several hundred thousand dollars, are designed around pharmaceutical or quality-control workflows, and typically dose one vial at a time from proprietary dosing heads.^{15,16} Loss-in-weight feeders from continuous pharmaceutical manufacturing solve a related problem at much larger mass flow rates,^{17–19} and laboratory micro-dosing studies have demonstrated mg-scale dosing with capillary, vibratory, and auger mechanisms,^{20–23} but none of these are available as open, reproducible, low-cost designs that a lab can print, assemble, and extend. The open-source hardware movement has shown that 3D-printed instruments can reach research-grade performance at 1–10% of commercial cost while enabling community-driven improvement,^{24–27} with platforms such as Jubilee and low-cost SDL toolkits demonstrating the pattern for automation hardware

^a Department of Mechanical Engineering, Brigham Young University, Provo, UT, USA. E-mail: swc19@student.byu.edu; sterling.baird@byu.edu

^b Acceleration Consortium, University of Toronto, 80 St George St, Toronto, ON M5S 3H6, Canada.

[†] Supplementary Information available: bill of materials, construction guide, auger exit-nozzle variants, and AI interaction logs. See DOI: 00.0000/00000000.

specifically.^{28,29}

This paper presents the base module of such an instrument: a single-channel, 3D-printed Archimedes-auger powder doser with tapping and vibration assistance, closed-loop gravimetric feedback, and a parts cost under \$200 (Fig. 1). The module is explicitly designed for replication—each channel is an independent printable unit that will later be arrayed around a shared collection cup to form a multi-powder doser (Section 4).

The second, equally deliberate contribution of this work is methodological. The platform’s mechanical parts were designed through generative artificial intelligence, principally an agent-mediated *programmatic* CAD workflow in which large-language-model (LLM) coding agents author parametric CAD code that humans review; end-to-end text-to-CAD services were evaluated separately as comparators, and one of them was adopted for late-stage parts. No conventional interactive CAD package (e.g. SolidWorks or Fusion 360) was used at any point from project start to finish: the only graphical CAD environment opened was Zoo Design Studio, introduced late in the project, driven primarily through chat prompts to its agent, and used in a limited, exploratory capacity until the final parts. Throughout, the division of labour was explicit—the human team made the design decisions, supplied specifications and drawings, reviewed every output, and printed and tested the parts, while the AI tools did the modelling—and per-part provenance (AI-generated, AI-revised, or human-corrected) is recorded in the repository design log. Generative CAD research has produced impressive datasets and models,^{30–33} and LLMs can emit CAD programs from natural-language prompts,^{34–37} but published evaluations focus on benchmark geometry rather than the messy reality of designing a working instrument end-to-end. Because the entire design history of this project—every prompt, every generated part, every review comment, and every failure—is preserved in a public GitHub repository, this platform doubles as a documented case study of AI-assisted hardware design.* We report the workflow that ultimately worked (part-by-part generation against explicit interface specifications, with automated geometry checks and human visual review), the recurring failure modes (assembly-level spatial reasoning, silent regressions, hallucinated part choices), and practical observations for others considering AI-assisted instrument design.

The objectives of this work are therefore: (1) deliver an open-source, low-cost, accurate single-powder dosing module suitable for SDL integration; (2) document and critically evaluate the use of generative AI as a primary CAD tool across a complete hardware project; and (3) establish the modular foundation—mechanical, electronic, and firmware—for the multi-powder doser, automated calibration, and powder-property database that constitute the subsequent stages of this program.

2 Results and discussion

2.1 Platform overview

Fig. 1 shows the single-channel module. There is no separate hopper: powder is loaded directly into the printed Archimedes auger (25 mm OD) through loading slots in the auger tube, so the auger itself is the powder reservoir. The auger is driven through a printed gear train by a NEMA-11 stepper motor. Dosing resolution is set by step-counted auger rotation; powder adhesion and bridging are broken by two auxiliary actuators: a 12 V push–pull solenoid striking a printed tap collar clamped to the auger tube, and an eccentric-rotating-mass (ERM) vibration motor mounted on the same collar. The auger assembly rides on a hinged mounting plate whose tilt is set by a servo-driven gear sector, so the auger can be parked horizontally (no gravity feed, clean shutoff) and tilted toward vertical for dispensing; the hinge geometry keeps the exit-nozzle dispense point stationary during tilt (Fig. 1c), so the dose lands in the same spot at any angle. Dispensed mass is measured by an analytical balance (A&D HR-100A, 0.1 mg readability) beneath the collection cup, read by the microcontroller over RS-232, closing the loop for coarse-then-trickle gravimetric dosing (Fig. 1d).

All structural parts are fused-filament printed in PLA with no support-critical geometry; non-printed components are commodity items (stepper, servo, solenoid, ERM motor, bearings, fasteners). The bill of materials totals under \$200 per channel excluding the shared analytical balance (SI Table S1), roughly two orders of magnitude below commercial gravimetric dosing heads. Control electronics are a single Raspberry Pi Pico W microcontroller running MicroPython, with a Tic T500 driver for the stepper, DRV8871 for the solenoid, DRV2605L for the vibration motor, and a MAX3232 transceiver for the balance’s RS-232 interface; the full schematic, firmware, and wiring guide are in the repository.

2.2 Design evolution

The mechanical design passed through four distinct generations in six weeks, all documented chronologically in a 97-entry design log in the repository (compressed in Fig. 1e). The project began as a mechanical scoop (“powder excavator”), hand-sketched by the human team, intended to ride an existing CNC gantry. Early viability analysis of dose control and powder adhesion, informed by AI-assisted literature surveys of micro-dosing mechanisms,^{20,21} prompted a structured comparison of eight dosing archetypes (tap-sieve, metering wheel, capillary wiper, auger, and others); the candidate enumeration was AI-generated, and the human team made the selection. The Archimedes auger won on dose resolution, printability, and powder generality, consistent with its dominance in commercial micro-feeders.^{15,16}

The second generation embedded the auger in a self-contained single-channel module (frame, drive, actuators) conceived by the human team as the repeating unit of a modular multi-powder architecture; the AI tools generated the corresponding CAD. The third generation was a deliberate methodological pivot: rather than asking the AI tools to generate the whole module at once, each part—auger, geared auger, bracket, tap col-

* All issue threads, pull requests, and design-log entries cited in this paper are preserved at <https://github.com/vertical-cloud-lab/powder-doser>.

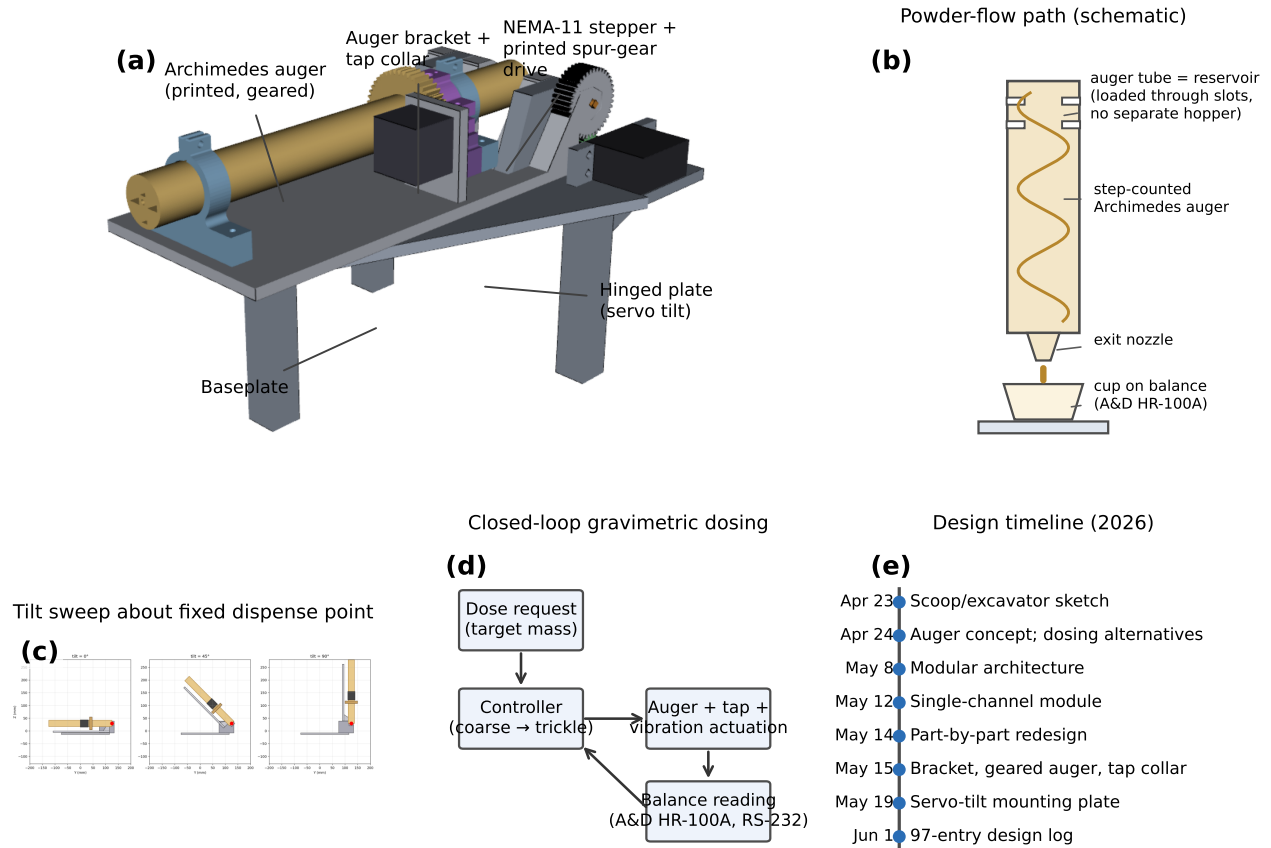


Fig. 1 Platform overview. (a) Single-channel powder-doser module (CAD render): printed Archimedes auger (25 mm OD), printed spur-gear drive from a NEMA-11 stepper, tap collar (which mounts the tap solenoid and ERM vibration motor, not individually resolved at this scale), and servo-tilted hinged mounting plate on the baseplate. (b) Schematic of the powder-flow path through the module: powder is loaded through slots into the auger tube, which is itself the reservoir (no separate hopper), and is conveyed down the helical channel → exit nozzle → collection cup on the analytical balance. (c) Tilt sweep about the fixed dispense point (red marker): the hinge axis passes through the exit-nozzle tip so the dispense point does not translate as the channel tilts from 0° (horizontal park) to 90° (vertical dispense). (d) Closed-loop gravimetric dosing concept: the target mass enters the controller, which drives the actuation while the balance feeds the measured mass back. (e) Compressed design timeline; the complete 97-entry design log (DESIGN-LOG.md) is available in the repository. Throughout, humans selected the architecture and reviewed, printed, and tested every part, while AI coding agents generated the parametric CAD models and renders; no GUI CAD package (e.g. Fusion 360, SolidWorks) was used.

lar, mounting plate, baseplate, servo hinge—was AI-modelled, human-reviewed, and printed individually against explicit interface specifications (Section 2.3). The specifications supplied to the AI grew progressively more prescriptive: the human team provided annotated engineering drawings, several of them fully dimensioned with geometric relations, so that part design went well beyond interface call-outs. By the late stages, the AI’s role had narrowed to modelling the parts—translating the drawings into parametric CAD and computing derived dimensions and tolerances—while all design decisions were made by the human team. The fourth generation integrated the printed parts on a hinged baseplate with servo tilt control and the fixed-dispense-point geometry of Fig. 1c. Key design specifics, including the auger tube cross-section and the split-clamp tap collar that couples solenoid impacts into the powder column without loading the auger bearing, are shown in Fig. 2. Exit-nozzle geometry strongly affects flow initiation and dribble; four printed nozzle variants are compared in SI Fig. S1.

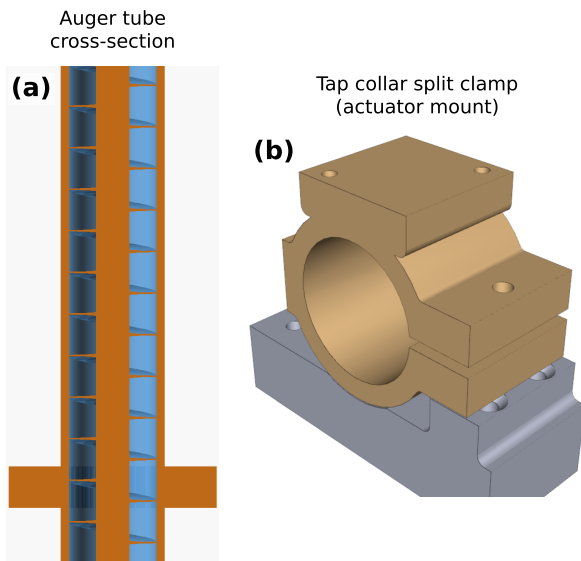


Fig. 2 Design specifics. (a) Cross-section of the printed auger tube showing the helical channel. (b) Split-clamp tap collar designed to carry the solenoid striker and ERM vibration motor; the clamp couples impacts into the powder column without loading the auger bearing.

2.3 Generative AI as a CAD tool

Our AI workflow used LLM coding agents (GitHub Copilot coding agent, primarily Claude-family models) operating directly on the version-controlled repository: the agent wrote parametric CAD as Python (build123d/CadQuery) or OpenSCAD code, rendered multi-view images and STLs in continuous integration, and opened pull requests that the human team reviewed using the rendered views. Two commercial text-to-CAD services, CADSmith and Zoo Design Studio—the latter pairing a code-based CAD kernel (KCL) with a conversational design agent, Zookeeper—were evaluated on the same parts for comparison, and Zoo Design Studio was ultimately adopted for late-stage parts (Tool comparison,

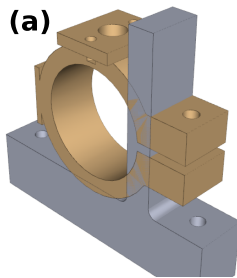
below). No conventional interactive CAD package was used at any point in the project; even within Zoo Design Studio, modelling was driven primarily through chat prompts rather than the graphical editor. Every prompt, generated file, render, and review comment is preserved in the repository’s pull requests and a dedicated discussion thread, satisfying emerging reproducibility guidance for LLM-assisted research.

2.3.0.1 Whole-assembly generation fails; part-by-part succeeds. Early attempts asked the agent to generate the complete module from an architecture description. The results were visually plausible but mechanically incoherent: the first tap collar was an unimplementable amalgamation of separate functions—interferences between features, incorrect tolerancing, and no space for the solenoid and vibration motor it exists to carry (Fig. 3a); an early bracket arrived with its two mounting plates not connected to each other; and assembly-level prompts repeatedly produced interferences, floating parts, and regressions, with feedback on one defect introducing new defects elsewhere. Restricting each generation task to a single part with explicitly enumerated interfaces (bolt patterns, shaft diameters, mating faces) converted the agent from a liability into a productive collaborator (Fig. 3c). This mirrors the known weakness of current LLMs in compositional spatial reasoning³⁴ and suggests that interface-first decomposition—standard practice in human mechanical design—is currently *mandatory* rather than optional for AI CAD. Across the 97 logged design iterations, each entry records its trigger, rationale, and outcome (built-and-worked, built-with-issues, failed, or superseded), giving per-part provenance and iteration counts; a structured quantitative analysis of this log (acceptance rates by workflow, defect taxonomy, and which defects were caught by automated checks versus human review) is reported in the repository and will accompany the multi-doser follow-up.

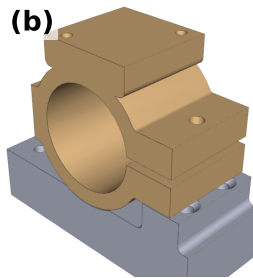
2.3.0.2 Trust, verification, and silent regressions. The dominant cost of the workflow was verification. Because any iteration could silently alter unrelated geometry, every generated part required full re-inspection; in one documented case the agent produced four miniature test nozzles with two of the four silently swapped relative to their documentation, which would have invalidated a dispensing comparison had it not been caught. Automated self-checks (single-body assertions, watertightness, interface-dimension probes run in CI) caught a useful subset of defects cheaply, but small-scale errors—a one-layer height mismatch discovered only when a print peeled off the build plate—escaped both the checks and three-view visual review. We observed a consistent division of labour across the design phase: generative exploration was genuinely valuable early, while final-stage convergence suffered from the agent’s inability to recognize how close a design was to its optimum, perturbing already-refined features. A corollary we observed repeatedly is that AI-designed parts plateau at “good enough”: the cost and risk of additional iterations discourages the final refinements a human designer would make by habit.

2.3.0.3 Tool comparison. The code-writing agent workflow (Copilot on the repository) was fully auditable and required no

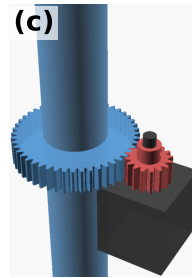
Tap collar, first AI proposal:
interferences, bad tolerancing,
no component clearance



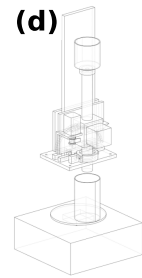
Tap collar after review iterations
(final part redesigned in Zoo)



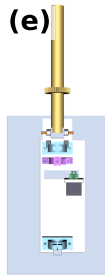
Geared auger + pinion
(part-by-part workflow)



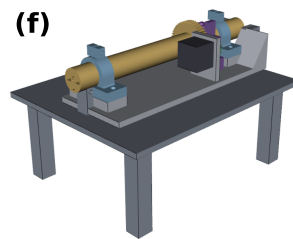
Whole-assembly attempt
(single prompt)



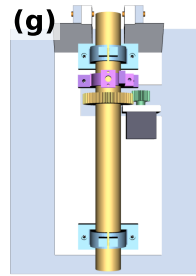
Iter. 1: unexplained
hole under gear



Iter. 2: raised platforms
instead of hole



Iter. 3: gap appears;
motor plate floats



Iter. 4: correct inputs
→ clean plate

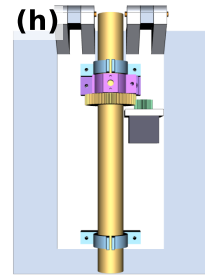


Fig. 3 Generative-AI CAD outcomes, good and bad. (a) First AI-generated tap collar: an amalgamation of separate functions with part-to-part interferences, incorrect tolerancing, no clearance for the solenoid and vibration motor it must carry, and a general lack of spatial reasoning—impossible to implement as drawn. (b) The same part after iterative review in the programmatic workflow; the production tap collar was subsequently redesigned in Zoo Design Studio (three iterations to a usable part). (c) Geared auger and stepper pinion produced by the part-by-part workflow. (d) A whole-assembly generation attempt (single prompt): plausible at a glance, but with interferences and floating components that took longer to diagnose than the part-by-part route took to build. (e–h) Four review iterations of the mounting plate: the agent inserted an unexplained hole (e), then raised platforms (f), then a gap that left the motor plate floating (g); only when the correct upstream part files were supplied did it produce a clean plate (h). The root cause was stale input files—a human-side error the agent silently designed around. All parts here were modelled by AI coding agents (programmatic CAD) and, for the production tap collar, the chat-driven Zoo Design Studio (Zookeeper agent); humans set the requirements and reviewed each iteration. No GUI CAD package was used.

software beyond a browser, at the cost of long feedback cycles (minutes per iteration) and review burden. CADSmith converged quickly on bracket- and plate-class parts once supplied with the same interface specifications. Zoo Design Studio, evaluated last, was judged by the design team to be the best of the tools tried: its Zookeeper agent exhibited markedly stronger spatial reasoning, ran built-in sanity checks (e.g. body counts) before returning results, and—because the agent operates inside a live, code-backed CAD view—produced editable models the team could inspect as they were generated. Individual iterations were slower (of order an hour), but one to two Zoo iterations surpassed what seven to eight Copilot iterations achieved, and the final tap collar reached a usable design in three Zoo iterations after the same number of Copilot iterations failed to orient the solenoid correctly. Its limits at evaluation time were the converse of its strengths: imported reference CAD was read-only, there were no multi-part assembly constraints, and manual edits in the graphical editor were largely restricted to parameter values, so interaction remained chat-driven. Two caveats temper these observations. First, the rankings are qualitative impressions from a single design campaign, not measurements; a quantitative cross-tool benchmark (identical prompts, identical parts, iteration counts and defect taxonomies) is in progress and will be reported with the multi-doser follow-up. Second, the team’s consistent impression was that every AI route was slower, and converged to lower-quality geometry, than experienced mechanical engineers working directly in traditional CAD would have been. What the AI-mediated workflow bought instead was thorough self-documentation as a side effect of working: every design intent, constraint, and revision exists as text, giving reproducibility, provenance, and transferability that conventional CAD sessions do not leave behind.

2.4 Cost and accessibility

The complete single-channel module—printed parts, motors, drivers, and microcontroller—costs under \$200 in single-unit quantities excluding the analytical balance, which most laboratories already own (SI Table S1), and prints in under 24 h on a consumer printer. For comparison, automated gravimetric dosing modules from commercial vendors are quoted at \$30k–\$300k depending on configuration. Even where commercial instruments outperform on absolute accuracy, the two-orders-of-magnitude cost gap changes what is possible: a lab can dedicate one printed channel per powder, accept cross-contamination-free disposable wetted parts, and replicate failed channels in a day. This is the economic logic of open-source laboratory hardware^{24,25} applied to the solid-handling gap in SDLs.

3 Experimental

3.1 Mechanical design and fabrication

All printed parts were produced in PLA on a Bambu Lab H2D (0.2 mm layers, 4 perimeters, 25% infill). The auger (25 mm OD printed helix, loaded through slots in the tube wall; no separate hopper), gear train (printed spur pair, 2.25:1), bracket, tap collar, mounting plate, hinged baseplate, and servo sector are parametric models; source (build123d/OpenSCAD/KCL), STLs, and ren-

ders for every iteration are in the repository. Non-printed components: NEMA-11 stepper (auger drive), MG996R-class servo (tilt), 12 V push–pull solenoid (tap), coin-type ERM motor (vibration), and 6805ZZ bearing (auger support). Gravimetric feedback uses an A&D HR-100A analytical balance (0.1 mg readability) under the collection cup.

3.2 Electronics and firmware

A single Raspberry Pi Pico W runs MicroPython firmware exposing dose primitives (rotate-by-steps, tap-burst, vibrate-burst, tilt-to-angle, read-mass). The stepper is driven by a Pololu Tic T500 over UART, the solenoid by a DRV8871 H-bridge, the vibration motor by a DRV2605L haptic driver over I²C; the A&D HR-100A balance is read over RS-232 through a MAX3232 transceiver. KiCad schematics, the wiring guide, and firmware are in `hardware/` in the repository.

3.3 AI design workflow and logging

LLM coding agents (GitHub Copilot coding agent; Claude-family models, 2026 sessions) executed design tasks specified as GitHub issues; each agent session produced a pull request containing generated CAD code, rendered orthographic/isometric views, and STLs, which the team reviewed in-browser. Literature and design-review queries were run through Edison Scientific; commercial text-to-CAD evaluations used CADSmith and Zoo Design Studio with the same part specifications, with Zoo Design Studio’s modelling performed through its Zookeeper conversational agent and adopted for late-stage parts. No conventional interactive CAD package was used at any stage of the project. In line with Digital Discovery’s LLM-usage guidance, the complete prompt/response history (issues, pull-request threads, agent session logs, and Zoo Design Studio transcripts), model identifiers, and generation dates are preserved in the public repository and summarized in the SI.

3.4 Dispensing characterization protocol

We define a gravimetric characterization protocol that ships with the firmware for evaluating a built channel. Doses are dispensed onto a tared collection cup on the analytical balance (A&D HR-100A, 0.1 mg readability), with $n \geq 10$ replicates per condition; a per-powder sweep of auger speed at fixed dose establishes mass-flow calibrations, and closed-loop accuracy is evaluated across 20 mg–5 g requested masses with coarse-then-trickle control. The controller dispenses in coarse mode (continuous rotation at the calibrated mass-flow rate) until 90% of the target mass is reached, then switches to trickle mode (single-step increments with tap/vibration bursts), sampling the balance’s RS-232 stream with a median-of-5 filter and accepting readings only in quiet windows ≥ 0.5 s after the last actuation; dosing terminates when the filtered mass reaches the target or a configurable overshoot/timeout bound trips. The design targets that motivated this control strategy—chosen so that dose error contributes less than one atomic percent of composition error in a typical five-component blend^{11,12}—are $\pm 5\%$ accuracy above 100 mg and $\pm 10\%$ at 20 mg, with per-dose times under 30 s. Controller pseu-

docode, parameters, and the characterization scripts ship with the firmware in the repository.

4 Conclusions

A single-channel, 3D-printed auger powder doser with closed-loop gravimetric control can be built for under \$200 from printed parts and commodity electronics, and its design process—conducted entirely through generative-AI tools under human review, without conventional CAD software—is fully documented and reusable. The methodological result is as transferable as the hardware: whole-assembly AI generation is not currently viable, but part-by-part generation against explicit interfaces, fortified by automated geometry checks and disciplined human review, is a practical and auditable way to design instruments. The complete, honest design history (97 logged iterations including failures) is itself a dataset for studying AI-assisted hardware design.

This base module is the first of five planned stages. Ongoing work extends the platform to (1) a multi-powder doser arraying N printed channels around a shared collection cup (Fig. 4), with a loaded subset of 8–12 powders and a path to a 50-powder pool via swappable sealed cartridges; (2) automated per-powder calibration that optimizes dosing parameters (speed, tap/vibration schedules) against the gravimetric loop; (3) a public database of optimized parameters across a large powder library, linking flowability descriptors to dosing behaviour; and (4) continued evaluation of generative tools, including AI-generated PCBs to replace the breadboard electronics. Each will be reported separately; the multi-doser geometry and the cartridge-sealing concepts already have working CAD in the repository.

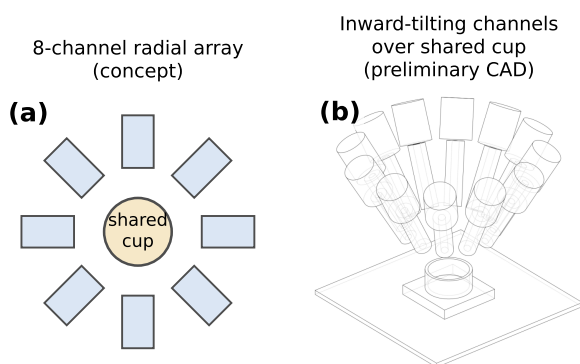


Fig. 4 Future work: multi-powder doser. (a) Eight-channel radial array concept (schematic) around a shared collection cup. (b) Preliminary programmatic-CAD study of inward-tilting channels over the shared cup. As elsewhere, the concept was set by the human team and the CAD modelled by AI coding agents; no GUI CAD package was used.

Author contributions

S.C.: conceptualization, methodology, investigation, CAD review, writing—original draft. W.M.: investigation, CAD review, validation. L.W.: investigation (electronics, PCB workflows), software. C.R.: investigation, hardware fabrication. G.E.: investigation, hardware fabrication. S.G.B.: conceptualization, supervision, funding acquisition, writing—review & editing.

Conflicts of interest

There are no conflicts to declare.

Data availability

All design files (parametric CAD source, STLs, renders), KiCad schematics, MicroPython firmware, AI prompt/response logs, the 97-entry design log, and the dispensing-characterization scripts are openly available at <https://github.com/vertical-cloud-lab/powder-doser>. Hardware designs are released under an open license (see repository). No experimental dispensing datasets are reported in this work. References cited in the SI are listed in this article's reference list.

Acknowledgements

Generative AI tools were used extensively and deliberately as research objects and assistants in this work: GitHub Copilot coding agent (Claude-family models) generated CAD code, firmware, and figure scripts under human review; Edison Scientific was used for literature search and design review; CADSmith and Zoo Design Studio (via its Zookeeper agent) were evaluated as text-to-CAD tools, with Zoo Design Studio adopted for late-stage part design. All AI usage is logged in the public repository. As an indication of the computational effort involved, the project consumed on the order of 6,700 premium GitHub Copilot requests—approximately 250 agent sessions triggered by the team across the repository's issues, pull requests, and discussions, at the $27\times$ request multiplier for the Claude Opus 4.8 model—and approximately 450 Edison Scientific credits (36 high-effort literature searches at 10 credits each, plus roughly 90 standard literature and analysis tasks at one credit each), tallied from the committed AI session logs and Edison response artifacts across all repository branches.

Notes and references

For the reference section, the style file `rsc.bst` can be used to generate the correct reference style.[§]

1. Citations should appear here in the format A. Name, B. Name and C. Name, *Journal Title*, 2000, **35**, 3523;
2. A. Name, B. Name and C. Name, *Journal Title*, 2000, **35**, 3523.

...

We encourage the citation of primary research over review articles, where appropriate, in order to give credit to those who first reported a finding. Find out more about our commitments to the principles of San Francisco Declaration on Research Assessment (DORA).

- 1 M. Abolhasani and E. Kumacheva, *Nature Synthesis*, 2023, **2**, 483–492.
- 2 G. Tom, S. P. Schmid, S. G. Baird, Y. Cao, K. Darvish, H. Hao, S. Lo, S. Pablo-García, E. M. Rajaonson, M. Skreta, N. Yoshikawa, S. Corapi, G. D. Akkoc, F. Strieth-Kalthoff,

- M. Seifrid and A. Aspuru-Guzik, *Chemical Reviews*, 2024, **124**, 9633–9732.
- 3 N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng and G. Ceder, *Nature*, 2023, **624**, 86–91.
 - 4 M. Seifrid, R. Pollice, A. Aguilar-Granda, Z. M. Chan, K. Hotta, C. T. Ser, J. Vestfrid, T. C. Wu and A. Aspuru-Guzik, *Accounts of Chemical Research*, 2022, **55**, 2454–2466.
 - 5 Y. Fei, B. Rendy, R. Kumar, O. Darts, H. P. Sahasrabudde, M. J. McDermott, Z. Wang, N. J. Szymanski, L. N. Walters, D. Milsted, Y. Zeng, A. Jain and G. Ceder, *ArXiv*, 2024.
 - 6 A. Aspuru-Guzik, *Digital Discovery*, 2023, **2**, 10–11.
 - 7 S. G. Baird, A. R. Falkowski and T. D. Sparks, *ArXiv*, 2026.
 - 8 P. M. Maffettone, P. Friederich, S. G. Baird, B. Blaiszik, K. A. Brown, S. I. Campbell, O. A. Cohen, R. L. Davis, I. T. Foster, N. Haghighi, M. Hereld, H. Joess, N. Jung, H.-K. Kwon, G. Pizzuto, J. Rintamaki, C. Steinmann, L. Torresi and S. Sun, *Digital Discovery*, 2023, **2**, 1644–1659.
 - 9 R. B. Canty, J. A. Bennett, K. A. Brown, T. Buonassisi, S. V. Kalinin, J. R. Kitchin, B. Maruyama, R. G. Moore, J. Schrier, M. Seifrid, S. Sun, T. Vegge and M. Abolhasani, *Nature Communications*, 2025, **16**, 3856.
 - 10 Z. Li, A. Ludwig, A. Savan, H. Springer and D. Raabe, *Journal of Materials Research*, 2018, **33**, 3156–3169.
 - 11 K. S. Vecchio, O. F. Dippo, K. R. Kaufmann and X. Liu, *Acta Materialia*, 2021, **221**, 117352.
 - 12 M. Moorehead, K. Bertsch, M. Niezgoda, C. Parkin, M. Elbakhshwan, K. Sridharan, C. Zhang, D. Thoma and A. Couet, *Materials & Design*, 2020, **187**, 108358.
 - 13 S. Mooraj, G. Kim, X. Fan, S. Samuha, Y. Xie, T. Li, J. S. Tile, Y. Chen, D. Yu, K. An, P. Hosemann, P. K. Liaw, W. Chen and W. Chen, *Communications Materials*, 2024, **5**, 14.
 - 14 P. Nelaturu, J. R. Hatrick-Simpers, M. Moorehead, V. Jambur, I. Szlufarska, A. Couet and D. J. Thoma, *Materials Science and Engineering: A*, 2024, **891**, 145945.
 - 15 M. Besenhard, E. Faulhammer, S. Fathollahi, G. Reif, V. Calzolari, S. Biserni, A. Ferrari, S. Lawrence, M. Llusà and J. Khinast, *European Journal of Pharmaceutics and Biopharmaceutics*, 2015, **94**, 264–272.
 - 16 S. Yang and J. Evans, *Powder Technology*, 2007, **178**, 56–72.
 - 17 W. E. Engisch and F. J. Muzzio, *Powder Technology*, 2012, **228**, 395–403.
 - 18 W. E. Engisch and F. J. Muzzio, *Powder Technology*, 2015, **283**, 389–400.
 - 19 J. Hanson, *Powder Technology*, 2018, **331**, 236–243.
 - 20 M. Besenhard, S. Fathollahi, E. Siegmund, E. Slama, E. Faulhammer and J. Khinast, *International Journal of Pharmaceutics*, 2017, **519**, 314–322.
 - 21 E. Faulhammer, M. Fink, M. Llusà, S. M. Lawrence, S. Biserni, V. Calzolari and J. G. Khinast, *International Journal of Pharmaceutics*, 2014, **473**, 617–626.
 - 22 S. Chianrabutra, B. Mellor and S. Yang, Proceedings of the Solid Freeform Fabrication Symposium, Austin, TX, 2014.
 - 23 J. Alsenz, *Powder Technology*, 2011, **209**, 152–157.
 - 24 J. M. Pearce, *HardwareX*, 2020, **8**, e00139.
 - 25 J. Hohlbein, B. Diederich, B. Marsikova, E. G. Reynaud, S. Holden, W. Jahr, R. Haase and K. Prakash, *Nature Methods*, 2022, **19**, 1020–1025.
 - 26 T. Wenzel, *PLOS Biology*, 2023, **21**, e3001931.
 - 27 J. Stirling, K. Bumke, J. Collins, V. Dhokia and R. Bowman, *International Journal of Computer Integrated Manufacturing*, 2022, **35**, 1297–1309.
 - 28 J. Vasquez, H. Twigg-Smith, J. T. O’Leary and N. Peek, Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems, 2020, pp. 1–13.
 - 29 B. Subbaraman, O. de Lange, S. Ferguson and N. Peek, *PLOS ONE*, 2024, **19**, e0296717.
 - 30 R. Wu, C. Xiao and C. Zheng, *Deepcad: a deep generative network for computer-aided design models*, 2021, <https://doi.org/10.48550/arxiv.2105.09492>, NEEDS MANUAL VERIFICATION: DOI returned non-404 but Crossref metadata unavailable.
 - 31 K. D. D. Willis, Y. Pu, J. Luo, H. Chu, T. Du, J. G. Lambourne, A. Solar-Lezama and W. Matusik, *ACM Transactions on Graphics*, 2021, **40**, 1–24.
 - 32 A. Seff, W. Zhou, N. Richardson and R. P. Adams, *Text*, 2021.
 - 33 X. Xu, J. Lambourne, P. Jayaraman, Z. Wang, K. Willis and Y. Furukawa, *ACM Transactions on Graphics*, 2024, **43**, 1–14.
 - 34 K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu and M. Gombolay, *ArXiv*, 2025.
 - 35 A. Badagabettu, S. S. Yarlagadda and A. B. Farimani, *ArXiv*, 2024.
 - 36 P. A. Jansen, *ArXiv*, 2023.
 - 37 T. Rios, S. Menzel and B. Sendhoff, 2023 IEEE Symposium Series on Computational Intelligence (SSCI), 2023, pp. 1704–1711.